

SEAS-MFEM Developer Guide

Component map, build reference, and change log

SEAS-MFEM team

2026-05-09

SEAS-MFEM Developer Guide

Document type: living developer onboarding map. Version: v0.1 (initial draft, 2026-05-09). Scope: entry points, module responsibilities, where to find the math, how to build & test, and how to navigate the supporting documents. Owners: anyone touching `miniapps/seas/`.

This guide is the *single page you start from*. It does not re-derive the weak forms or numerical schemes — those live in `CODEBASE_GUIDE.md` (physics-to-numerics walkthrough) and `ARCHITECTURE.md` (class hierarchy / data flow). This document tells you which file to open and which deeper document to read for any task.

Quick-Access Cheatsheet

All file paths are relative to `miniapps/seas/`. Long paths in the middle column are wrapped at directory boundaries.

If you want to ...	Open this first	Then read
Understand the simulation pipeline end-to-end	<code>drivers/ seas_driver.cpp</code>	§3 Drivers ; <code>ARCHITECTURE.md</code> Exec Flow
Modify the DG weak form / penalty / fault flux	<code>integrator/ dg_elasticity_ip_combined_ integrator.hpp</code>	§5 Domain ; <code>CODEBASE_GUIDE.md</code> §6, §15
Add a friction law or state-evolution law	<code>friction/friction_law.hpp, friction/state_evolution.hpp</code>	§7 Friction ; <code>CLAUDE.md</code> “Friction Solver”
Change BP5 spatial parameters $a(x_2, x_3), D_c, \tau_0$	<code>config/bp5_params.hpp</code>	§4 Config ; <code>CLAUDE.md</code> extreme-care list
Touch fault DOF management, slip embedding, traction proj.	<code>fault/fault_basis.hpp, fault/fault_geometry.hpp</code>	§6 Fault layer ; <code>CODEBASE_GUIDE.md</code> §7
Work on dynamic rupture (TPV102 / 104 / 205)	<code>dynamic/wave_operator.hpp, dynamic/fault_face_flux.hpp</code>	§9 Dynamic rupture
Work on the SAFS (San Andreas) mesh pipeline	<code>safs/project_7.0_preferred/ code_meshing/</code>	§10 SAFS
Add an MPI collective or debug a parallel hang	<code>common/mpi_context.hpp, common/fault_scatter.hpp</code>	§11 Parallelism ; <code>CODEBASE_GUIDE.md</code> §11
Change adaptive time stepping or RK45 tolerances	<code>solver/time_stepper.hpp; drivers/seas_driver.cpp</code> Stage 7	§8 Solver
Add a unit / parallel / verification test	<code>tests/{unit,parallel, verification}/ + Makefile</code>	§12 Build & test
Trace an output column to its source	<code>io/bp5_parallel_output.hpp (BP5); io/paraview_output.hpp</code>	§13 I/O
Audit a numerical bug-fix history	<code>debug_document/ <problem>_debug_document/</code>	<code>CLAUDE.md</code> “Before ANY Code Change”

Reading order for new contributors: this guide -> ARCHITECTURE.md -> the relevant section of CODE-BASE_GUIDE.md. Read CLAUDE.md before editing any file in the “extreme-care” list (see §14).

Table of Contents

1. Repository Layout
2. Symbol & Notation Glossary
3. Drivers (Entry Points)
4. Configuration System
5. Domain Layer (DG Elasticity / Antiplane)
6. Fault Layer (DOF Management & Geometry)
7. Friction & State Evolution
8. Solver & Time Integration (Quasi-Dynamic)
9. Dynamic Rupture Module (dynamic/)
10. SAFS Mesh Pipeline (safs/)
11. Parallelism & MPI
12. Build & Test Reference
13. I/O, Output & Visualization
14. Files Requiring Extreme Care
15. Document Map (Where Things Live)
16. Change Log

1. Repository Layout

```
miniapps/seas/
├── drivers/
│   ├── seas_driver.cpp      # Entry points: one driver per benchmark family
│   ├── tpv102_driver.cpp    # Quasi-dynamic, TOML-configured (BP1/BP2/BP5)
│   ├── tpv104_driver.cpp    # Dynamic rupture, SCEC TPV102 (rate-state aging)
│   ├── tpv104_driver.cpp    # Dynamic rupture, SCEC TPV104 (slip-law-SRW)
│   └── tpv205_driver.cpp    # Dynamic rupture, SCEC TPV5/205 (linear slip-weakening)
├── pseas.cpp                # Legacy parallel BP2 driver (hardcoded, antiplane)
├── config/                  # TOML config + per-benchmark parameter packs
├── constitutive/            # Material models (currently isotropic linear elastic)
├── domain/                  # Quasi-dynamic DG elasticity / antiplane PDE solvers
├── integrator/              # MFEM bilinear-form integrators (IP / BR2)
├── fault/                   # Fault DOF management, basis, quadrature
├── friction/                # Friction laws (regularized Dieterich-Ruina, slip-law-SRW)
├── solver/                  # Quasi-dynamic SEAS coupling + Dormand-Prince RK45
├── dynamic/                 # Δ DYNAMIC RUPTURE (separate from quasi-dynamic stack)
│   ├── wave_operator,      # wave_operator, godunov_flux, ADER substep iterators,
│   ├── per-TPV setup /     # per-TPV setup / friction / nucleation modules.
│   └── friction /          #
├── io/                      # Probe writers, ParaView, checkpoint, sidecar fields
├── common/                  # MPI context, fault scatter, types, logging
├── trace/                   # Per-face runtime tracer (rank-local, barrier-free)
├── bp1/, bp2/, bp5/         # Per-benchmark mesh + reference data + plots
├── tpv102/, tpv104/, tpv205/ # Per-benchmark mesh + gold + plots
├── safs/                    # Δ SAN ANDREAS FAULT SYSTEM mesh pipeline (Python)
├── tests/                   # unit/, parallel/, verification/, scripts/
└── jobs/                    # SLURM sbatch scripts (Frontera, by benchmark)
```

```

├─ tools/                # One-off utilities (mesh generators, etc.)
├─ document/             # Plans, debug history, this guide
├─ debug_document/       # Per-problem chronological debug logs (READ FIRST)
├─
├─ extern/toml11/        # Vendored TOML parser
├─ ARCHITECTURE.md       # Class hierarchy, data flow, exec model
├─ CODEBASE_GUIDE.md     # Physics-to-numerics walkthrough (DG derivation)
├─ CLAUDE.md             # Critical numerical constraints + extreme-care list
├─ DEVELOPER_GUIDE.md    # ← THIS FILE (high-level component map)
├─ Makefile              # Primary build system (CMakeLists.txt also exists)

```

Mental model: there are two distinct physics stacks sharing mesh, config, friction, fault/fault_basis, common, and io:

- Quasi-dynamic (domain/ + integrator/ + solver/ + seas_driver.cpp): solves the static elasticity BVP at each ODE evaluation; couples to a rate-and-state ODE through radiation damping.
- Dynamic rupture (dynamic/ + the tpv*_driver.cpp family): solves the first-order velocity-stress wave equation with ADER time stepping; fault interface is treated by a Riemann/Godunov solver instead of DG-IP penalties.

These stacks do not share **domain/elasticity_operator.hpp**. Be deliberate about which world you are in before editing a file.

2. Symbol & Notation Glossary

Continuum / DG

Symbol	Meaning	Units
$\mathbf{u}(\mathbf{x}, t)$	Displacement field	m
$\boldsymbol{\varepsilon} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^\top)$	Small-strain tensor	—
$\boldsymbol{\sigma} = \mathbb{C} : \boldsymbol{\varepsilon} = \lambda \operatorname{tr}(\boldsymbol{\varepsilon}) \mathbf{I} + 2\mu \boldsymbol{\varepsilon}$	Cauchy stress	Pa
λ, μ	Lamé parameters	Pa
$c_s = \sqrt{\mu/\rho}, c_p = \sqrt{(\lambda + 2\mu)/\rho}$	S- / P-wave speed	m/s
ρ	Mass density	kg/m ³
$\Omega, \Gamma^D, \Gamma^N, \Gamma^F, \Gamma^0$	Domain; Dirichlet, Neumann, fault, interior non-fault faces	—
$\mathbf{n}$	Outward unit normal (from K^- to K^+ on interior faces)	—
$\{\{q\}\} = \frac{1}{2}(q^- + q^+)$	Face average	—
$\llbracket q \rrbracket = q^- - q^+$	Face jump	—
η_e/h_e	DG-IP penalty (per-face)	varies
$\boldsymbol{\delta}$	Prescribed fault slip vector ($\llbracket \mathbf{u} \rrbracket$ on Γ^F)	m
$\mathbf{g}^D(\mathbf{x}, t)$	Prescribed Dirichlet displacement	m

Fault & friction

Symbol	Meaning	Units
$V = \dot{\boldsymbol{\delta}}, \mathbf{V}$	Slip rate (scalar magnitude / vector)	m/s
V_p	Plate (loading) rate	m/s
V_0	Reference slip rate (Dieterich-Ruina)	m/s
V_{init}	Initial fault slip rate	m/s
V_{nuc}	Nucleation patch slip rate (BP5)	m/s
a, b	Direct-effect / state-evolution coefficients	—
D_c (a.k.a. L, L_0)	Critical slip distance	m

Symbol	Meaning	Units
f_0	Reference friction coefficient	—
$f(V, \psi)$	Regularized rate-state friction coefficient	—
ψ	Logarithmic state variable; $\psi = f_0 + b \ln(V_0 \theta / D_c)$	—
θ	“Aging” state variable (alternative to ψ)	s
σ_n	Effective normal stress (> 0 = compression)	Pa
$\eta = \mu / (2c_s)$	Radiation-damping coefficient	Pa·s/m
τ, τ_{pre}	Total / pre-stress shear traction (fault-local)	Pa
$\mathbf{T}$	DG-recovered traction vector (global frame)	Pa
$\hat{\mathbf{n}}, \hat{\mathbf{d}}, \hat{\mathbf{s}}$	Fault unit normal / dip / strike (FaultBasis)	—
W_f, l_f	Fault depth / length	m
$H, l_{\text{vw}}, w_{\text{nuc}}$	VW-region depth / strike length / nucleation width	m

Numerics

Symbol	Meaning
K	Element (mesh cell)
p	Polynomial order
$J_K, \det J_K $	Element Jacobian, its determinant
$\Delta t, \Delta t_{\text{init}}, \Delta t_{\text{max}}$	Time step (current / initial / max)
atol, rtol	Absolute / relative ODE tolerances
$\mathbf{Q}$	Velocity-stress state vector ($\sigma_{xx}, \dots, v_z$) $\in \mathbb{R}^9$ (dynamic)
$A_n^\pm$	Split flux Jacobians (Godunov flux, dynamic)

Coordinate frames

- Mesh / code: X = along-strike, Y = fault-normal (fault at $Y = 0$), Z = vertical (surface at $Z = 0$, depth $Z < 0$).
- SCEC convention (used in benchmark spec): x_1 = fault-normal, x_2 = along-strike, x_3 = depth-positive-down. Mapping: $x_1 \leftrightarrow Y$, $x_2 \leftrightarrow X$, $x_3 \leftrightarrow -Z$.
- Fault-local frame (FaultBasis, BP5/Tandem canonical, project-wide): tangent1 = dip, tangent2 = strike. So in DOfData, $V1/\text{slip1}/\text{tau1}_0$ are dip components and $V2/\text{slip2}/\text{tau2}_0$ are strike. See CLAUDE.md “Sign Conventions” for the full chain.

3. Drivers (Entry Points)

Driver	Physics	Time stepper / Output
drivers/ seas_driver.cpp	Quasi-dynamic 3D elasticity + RS friction (BP5, BP1, BP2 via TOML). Mesh: <code>--mesh <gmsh.msh></code> or inline.	PETSc TS (rk5dp) default; native DOPRI5 fallback. SCEC <code>fltst</code> , ParaView, checkpoints.
pseas.cpp (legacy)	Quasi-dynamic 2D antiplane (BP2); hardcoded inline mesh.	Native DOPRI5. Probe ASCII.
drivers/ tpv102_driver.cpp	Dynamic rupture, regularized RS aging-law (SCEC TPV102). Mesh: <code>tpv102/mesh/*.msh</code> .	ADER substep (per-face order O). SCEC fault + free-surface stations.
drivers/ tpv104_driver.cpp	Dynamic rupture, slip-law-with-SRW (SCEC TPV104, FL=103). Mesh: <code>tpv104/mesh/*.msh</code> .	ADER substep with cumulative nucleation accumulator. SCEC TPV104 station format.
drivers/ tpv205_driver.cpp	Dynamic rupture, linear slip- weakening (SCEC TPV5 / 205). Mesh: <code>tpv205/mesh/*.msh</code> .	ADER substep, closed-form LSW solve (no ψ). SCEC TPV5 station format.

seas_driver.cpp — 9-stage pipeline

1. Parse TOML + CLI overrides -> SEASConfig
2. Load Gmsh mesh -> ParMesh (MPI partition)
3. Construct ConstitutiveModel + ElasticityDomainOperator (assembles K once)
4. Build FaultGeometry, DieterichRuinaFriction, AgingLawPsi -> RateStateFaultOperator
5. Wrap in SEASQuasiDynamicOperator + 4-phase init
6. Set up I/O: ParallelBP5BenchmarkOutput + global V_max log
7. Configure time stepper: PETSc TS (default) or native DormandPrinceRK45
8. Run time loop with adaptive dt + earthquake detection + checkpointing
9. Force-write final state and summary

The Mult(state, rate) evaluation called at every RK stage performs the core physics cycle:

$$\underbrace{\text{slip}}_{\text{state}} \xrightarrow{\text{ghost exchange}} \underbrace{K\mathbf{u} = \mathbf{b}_{\text{slip}} + \mathbf{b}_{\text{Dir}}}_{\text{linear solve}} \xrightarrow{\text{DG flux}} \mathbf{T} \xrightarrow{\text{Brent}} \underbrace{(\mathbf{V}, \dot{\psi})}_{\text{rate}}$$

with the fault stress balance (quasi-dynamic):

$$|\tau_{\text{pre}} + \mathbf{T}_{\text{fault-local}}| = \sigma_n f(|\mathbf{V}|, \psi) + \eta |\mathbf{V}|.$$

Step-by-step in code: solver/seas_operator.hpp::Mult (see CODEBASE_GUIDE.md §2 for the annotated walkthrough).

TPV* drivers — common shape

All three TPV drivers share the same skeleton (deliberately *not* refactored into a base class — see §14):

1. Build ParMesh + custom fault-locality partition (dynamic/fault_locality_partition.hpp)
2. Construct WaveOperator<ParMesh> (velocity-stress DG, Godunov flux, optional PML)
3. Construct FaultFaceFlux + per-TPV substep iterator
4. Initialize per-DOF DOFData (impedances, pre-stress, a/b/Dc, ψ_{init} , V_w side-channel)
5. ADER time loop: macro-step predictor -> per-substep friction solve -> state update
6. Per-step station output + ParaView snapshots

The substituted pieces between TPV102/104/205 are exactly the friction law, the state-evolution analytic update, and the nucleation accumulator (the file headers in dynamic/tpv*_setup.hpp enumerate the diffs explicitly).

4. Configuration System

Pipeline: TOML file -> CLI overrides (--key=value) -> SEASConfig struct -> SEASConfigBridge -> typed runtime objects (BP5Params, BoundaryConfig, DomainConfig, ...) -> operator constructors.

File	Role
config/seas_config.hpp	SEASConfig aggregate (no logic, just nested structs)
config/seas_config_parser.hpp	TOML -> SEASConfig, CLI overrides, defaults, validation
config/seas_config_bridge.hpp	SEASConfig -> BP5Params / BoundaryConfig / DomainConfig
config/bp1_params.hpp, bp2_params.hpp, bp5_params.hpp	Per-benchmark physical constants and spatial functions $a(x_2, x_3)$, $D_c(x_2, x_3)$, $\tau_0(x_2, x_3)$, $V_{\text{init}}(x_2, x_3)$
config/tpv102_params.hpp, tpv104_params.hpp, tpv205_params.hpp	Per-TPV material + friction constants matching the SCEC PDF spec
config/bp5_mesh_utils.hpp	Inline mesh creation (smoke tests)

Material constants flow: `MaterialConfig` derives Lamé parameters from ρ, c_s, ν :

$$\mu = \rho c_s^2, \quad \lambda = \frac{2\nu\mu}{1-2\nu}.$$

Per-DOF spatial parameters (BP5 example): `FaultGeometry::ComputeBP5Params` walks every fault DOF, evaluates `bp5_params_.a_of_x2_x3(x2,x3)`, etc., and stores the result. Never hardcode these constants in operator code — they must come from `*_params.hpp` (see §14 feedback rule).

5. Domain Layer (DG Elasticity / Antiplane)

Quasi-dynamic stack only. For dynamic rupture see §9.

Class hierarchy

```
DomainOperator<MeshType> [domain/domain_operator.hpp]
| Solve(t, slip, u); ComputeTraction(u, slip, T);
| RestrictToOwnedFault, ExpandOwnedToLocalFault, GetFaultBasis
|— ElasticityDomainOperator<MeshType> [domain/elasticity_operator.hpp]
|   3D vector DG (IP or BR2) + ConstitutiveModel
|— AntiplaneDomainOperator<MeshType> [domain/antiplane_operator.hpp]
|   2D scalar Laplace (BP1/BP2)
```

Strong form (3D linear elasticity, quasi-static)

$$-\nabla \cdot \sigma = 0 \text{ in } \Omega, \quad \sigma n = T \text{ on } \Gamma^N, \quad u = g^D \text{ on } \Gamma^D, \quad \llbracket u \rrbracket = \delta \text{ on } \Gamma^F.$$

DG-IP weak form (the version assembled by the integrator)

Find $u \in V_h$ such that for all $v \in V_h$, $a(u, v) = L(v)$ with

$$\begin{aligned} a(u, v) = & \sum_K \int_K \nabla v : \mathbb{C} : \nabla u \, dx \\ & - \sum_{e \in \Gamma^0 \cup \Gamma^F} \int_e (\llbracket u \rrbracket \cdot \{\{\mathbb{C} : \nabla v\}\} n + \llbracket v \rrbracket \cdot \{\{\mathbb{C} : \nabla u\}\} n) \, ds \\ & + \sum_{e \in \Gamma^0 \cup \Gamma^F} \int_e \frac{\eta_e}{h_e} \llbracket u \rrbracket \cdot \llbracket v \rrbracket \, ds \\ & - \sum_{e \in \Gamma^D} \int_e (u \cdot (\mathbb{C} : \nabla v) n + v \cdot (\mathbb{C} : \nabla u) n) \, ds + \sum_{e \in \Gamma^D} \int_e \frac{\eta_e}{h_e} u \cdot v \, ds, \\ L(v) = & - \sum_{e \in \Gamma^D} \int_e g^D \cdot (\mathbb{C} : \nabla v) n \, ds + \sum_{e \in \Gamma^D} \int_e \frac{\eta_e}{h_e} g^D \cdot v \, ds \\ & - \sum_{e \in \Gamma^F} \int_e \delta \cdot \{\{\mathbb{C} : \nabla v\}\} n \, ds + \sum_{e \in \Gamma^F} \int_e \frac{\eta_e}{h_e} \delta \cdot \llbracket v \rrbracket \, ds. \end{aligned}$$

Full derivation (strong -> flux form -> primal form -> IP fluxes) is in `CODEBASE_GUIDE.md` §15. The IP penalty parameter formula (p -dependent, includes the elasticity-tensor coupling ratio $= (d\lambda + 2\mu)^2 / (2\mu)$) is in `CODEBASE_GUIDE.md` §6.2.1.

Integrators

File (under integrator/)	Method	Used for
dg_elasticity_ip_combined_integrator.hpp	SIPG; one integrator does $K + \text{slip-RHS} + \text{traction}$	3D elasticity, default IP path
dg_elasticity_br2_integrator.hpp	Bassi-Rebay 2 lifting	3D elasticity, alt.
dg_elasticity_ip_penalty_integrator.hpp	Penalty-only (full elasticity tensor)	Legacy / split assembly
dg_br2_integrator.hpp	Scalar BR2	2D antiplane

The combined IP integrator deliberately fuses stiffness, slip-RHS, and traction recovery so they share the *same* traction operator and quadrature rule (K-b consistency).

DG traction recovery (the input to friction)

$$\mathbf{T}(\mathbf{x}_q) = \underbrace{\frac{1}{2}(\sigma^- + \sigma^+) \hat{\mathbf{n}}}_{\text{stress average}} - \underbrace{\alpha (\llbracket \mathbf{u} \rrbracket - \delta)}_{\text{penalty correction}}.$$

Per-DOF values are obtained by Galerkin L^2 projection of $\mathbf{T}(\mathbf{x}_q)$ onto the face basis (fault/face_quadrature.hpp::GalerkinProject), then rotated to fault-local coordinates by FaultBasis::ProjectTraction.

6. Fault Layer

File	Role
fault/rate_state_fault.hpp	RateStateFaultOperator<MeshType, SlipComponents> — owns the ODE state vector, PreInit / Init / ComputeRHS
fault/fault_geometry.hpp	Per-DOF $a, D_c, \eta, \tau_{\text{pre}}, V_{\text{init}}$, depths, owned/ghost masks
fault/fault_basis.hpp	Per-face $(\hat{\mathbf{n}}, \hat{\mathbf{d}}, \hat{\mathbf{s}})$ frames; embed slip / project traction
fault/face_quadrature.hpp	Multi-DOF face basis + L^2 projection (used at $p \geq 2$)
fault/fault_nodes.hpp	Fault DOF node management

State vector layout

- Antiplane (SlipComponents=1, StatePerNode=2): state = $[\delta_0, \psi_0, \delta_1, \psi_1, \dots]$
- 3D elasticity (SlipComponents=2, StatePerNode=3): state = $[\delta_{\text{dip},0}, \delta_{\text{strike},0}, \psi_0, \dots]$

FaultBasis orientation rule (the subtle bit)

For each face the basis is built from a raw mesh normal $\mathbf{n}_{\text{raw}}$ and a reference normal $\mathbf{n}_{\text{ref}}$:

1. If $\mathbf{n}_{\text{raw}} \cdot \mathbf{n}_{\text{ref}} < 0$, set **sign_flipped** = true and reverse $\mathbf{n}_{\text{raw}}$.
2. $\hat{\mathbf{n}} = \mathbf{n}_{\text{raw}} / \|\mathbf{n}_{\text{raw}}\|$, $\hat{\mathbf{s}} = \mathbf{up} \times \hat{\mathbf{n}}$ (with $\mathbf{up} = (0, 0, -1)$), $\hat{\mathbf{d}} = \hat{\mathbf{s}} \times \hat{\mathbf{n}}$.
3. If **sign_flipped**, negate all three vectors.

The stored vectors already encode the DG-jump sign, so callers (EmbedSlip, ProjectTraction) use them with no additional sign logic. See CODEBASE_GUIDE.md §7.3 for the *why*.

7. Friction & State Evolution

File	Class	Used by
friction/friction_law.hpp	abstract FrictionLaw	all friction sites

File	Class	Used by
friction/dieterich_ruina.hpp	DieterichRuinaFriction (Brent solver, $\log_{10} V$ space)	BP5 + TPV102
friction/state_evolution.hpp	AgingLaw{,Psi}, SlipLaw{,Psi}	quasi-dynamic + TPV102
friction/slip_law_srw_psi.hpp	SlipLawSRWPsi (FVW / SCEC FL=103)	TPV104 dynamic
friction/friction_coeff_stable.hpp	Branch-protected asinh($x e^c$) for large ψ/a	TPV104 dynamic

Regularized Dieterich-Ruina

$$f(V, \psi) = a \operatorname{asinh} \left[\frac{V}{2V_0} \exp \left(\frac{\psi}{a} \right) \right].$$

Stress balance to solve for V (quasi-dynamic, scalar form):

$$\tau = \sigma_n f(V, \psi) + \eta V.$$

Solver: Brent's method on $F(V) = \tau - \sigma_n f(V, \psi) - \eta V$ in $\log_{10}(V)$ space. Newton fails at large ψ/a (see debug v1). Initial bracket: $[V_a = 10^{-32}, V_b = \tau/\eta]$, with a fallback $V_a = 10^{-300}$ for extreme ψ . Do not change to Newton.

State evolution (aging law in ψ -space, BP5 default)

$$\dot{\psi} = \frac{bV_0}{D_c} \left[\exp \left(\frac{f_0 - \psi}{b} \right) - \frac{V}{V_0} \right], \quad \psi_{ss} = f_0 + b \ln(V_0/V).$$

For TPV104, the analytic ψ update follows the slip-law-with-SRW formulation in `slip_law_srw_psi.hpp`. For TPV205, there is no state variable — friction is linear slip-weakening:

$$\mu_{\text{eff}}(\delta) = \mu_s - (\mu_s - \mu_d) \min \left(\frac{\delta}{d_c}, 1 \right), \quad |V| = \max \left(0, \frac{|\tau_{\text{total}}| - \mu_{\text{eff}} \sigma_n}{\eta_s} \right).$$

8. Solver & Time Integration (Quasi-Dynamic)

File	Class	Role
solver/seas_operator.hpp	SEASQuasiDynamicOperator<MeshType, DomainOpT, FaultOpT>	Couples domain + fault into one TimeDependentOperator; Mult(state, rate) runs one ODE-RHS evaluation; SetInitialCondition runs the 4-phase init
solver/time_stepper.hpp	DormandPrinceRK45	7-stage embedded pair (5(4)), FSAL, PI controller, MPI-aware error reduction, V-guard
solver/seas_bdrload_operator.hpp	BP1-style boundary-load coupling	BP1 only

Defaults (BP5): $\text{atol} = 1\text{e-}7$, $\text{rtol} = 1\text{e-}50$ (pure absolute); $\Delta t_{\text{init}} = 0.01 L_{\text{nuc}}/V_{\text{nuc}}$; $\Delta t_{\text{max}} = 0.1 \text{ yr}$; V-guard factor 100.

4-phase initialization (SetInitialCondition):

1. PreInit: $\delta = 0$, $\psi = \psi_{ss}(V_{\text{init}}, D_c)$.
2. First domain solve with zero slip -> initial elastic traction.
3. Init: invert $|\tau_0 + T_{\text{elastic}}| = \sigma_n f(V_{\text{init}}, \psi_0) + \eta V_{\text{init}}$ for ψ_0 .
4. Verification re-solve — equilibrium error must drop below 10^{-6} .

PETSc TS (rk5dp + TSADAPT_BASIC) is the default time stepper; the native DormandPrinceRK45 is the fallback when PETSc is not available. Both share Mult.

9. Dynamic Rupture Module (**dynamic/**)

A self-contained, first-order velocity-stress DG solver with explicit ADER time stepping. Used by the TPV102/104/205 drivers; not wired into `seas_driver.cpp`.

State vector

$$\mathbf{Q} = (\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}, v_x, v_y, v_z)^\top \in \mathbb{R}^9$$

with index aliases in `dynamic/wave_state.hpp::QIndex`.

Module map

File	Role
<code>dynamic/wave_operator.{hpp,cpp,inl}</code>	<code>WaveOperator<MeshType></code> — DG bulk operator, ADER predictor, Mixed-Flux modes (Zhang 2023), free-surface BC modes (Gamma / Godunov)
<code>dynamic/godunov_flux.{hpp,cpp}</code>	Split-flux Jacobians $A_n^\pm$ (de la Puente 2009 / SeisSol parity)
<code>dynamic/precomputed_face_fluxes.{hpp,cpp}</code>	Per-face precomputed $A_n^\pm$ caches
<code>dynamic/pml_layer.{hpp,cpp}</code>	Convolutional PML (Komatitsch & Martin 2007), per-face directional damping
<code>dynamic/fault_face_flux.{hpp,cpp}</code>	<code>FaultFaceFlux</code> — Riemann-style fault flux + per-DOF <code>DOFData</code> (impedances, pre-stress, ψ , V)
<code>dynamic/friction_solver.{hpp,cpp}</code>	Dual Brent (CPU) / Newton (GPU) friction solve; reuses <code>DieterichRuinaFriction</code> for the Brent path
<code>dynamic/heterogeneous_material.hpp</code>	<code>MaterialField</code> adapter (constant or <code>ParGridFunction</code> -backed (λ, μ, ρ)) — Phase 5 of data-projection
<code>dynamic/fault_locality_partition.hpp</code>	Forces both elements of every fault face onto the same MPI rank (G1 fix)
<code>dynamic/seas_dynamic_operator.hpp</code>	Thin <code>TimeDependentOperator</code> wrapper around <code>WaveOperator</code> + <code>FaultFaceFlux</code>
<code>dynamic/seas_diag_rank.hpp</code>	Rank-local diagnostics
<code>dynamic/d4_tet_mesh.hpp</code>	D_4 -equivariant tet fixture (test-only)
<code>dynamic/shared_fault_key.hpp</code>	Stable face IDs across MPI ranks
<code>dynamic/tpv102_setup.hpp</code>	TPV102: per-DOF init, nucleation, station writers
<code>dynamic/tpv102_nucleation.hpp</code>	TPV102 boxcar nucleation accumulator
<code>dynamic/tpv102_friction_solver.hpp</code>	TPV102-specific friction wrapper
<code>dynamic/tpv102_substep_iterator.{hpp,cpp}</code>	Per-ADER-substep friction + ψ + nucleation orchestrator
<code>dynamic/tpv104_*</code>	TPV104 analogs (slip-law-SRW, FVW init, V_w side-channel, cumulative nucleation)
<code>dynamic/tpv205_*</code>	TPV205 analogs (LSW friction, patch pre-stress, no ψ , no nucleation accumulator)
<code>dynamic/tpv102_setup_total.hpp</code>	Total-Q variant of TPV102 init (background prestress lives in bulk $\mathbf{Q}$)

Per-substep loop (TPV104, representative)

For each ADER substep $o \in \{0, \dots, O - 1\}$:

1. Nucleation increment at the substep endpoint: $\tau_{2,\text{nuc}} += \Delta\tau_{\text{nuc}}^{(o)}$
2. **FaultFaceFlux::ComputeStageState** on $\bar{Q} = I/\Delta t$ — solves friction.
3. Analytic ψ update for $\Delta t_{\text{sub}}^{(o)}$ via `UpdateStateAnalyticSlipLawSRW`.
4. **BuildImposedState** -> per-substep $Q_{\text{imp}}^{\pm,(o)}$.
5. Accumulate $I_{\text{imp}}^{\pm} += w^{(o)} \Delta t_{\text{macro}} Q_{\text{imp}}^{\pm,(o)}$.

After the last substep, `WriteBackState` records `slip_rate`, V_1 , V_2 , τ_{corr}^* , σ_n^{corr} on `DofData` for probe output. At $O = 1$ this collapses bit-identically to the legacy single-stage `EvaluateADER` path.

Critical **FaultFaceFlux** files

`dynamic/fault_face_flux.{hpp,cpp}` and `dynamic/wave_operator.{hpp,inl}` are shared between TPV102, TPV104, TPV205, and BP5 dynamic paths — they are on the do-not-touch list (§14). Per-TPV behavior is added by the substep iterator and setup headers, not by editing these.

Relevant plans

- `document/system_dev/dynamic_rupture_plan_v{1,2,3,4}.md` — design history
- `document/system_dev/tpv205_lsw_native_fields_plan_2026-04-27.md` — TPV205 LSW design
- `document/system_dev/drdg3d_mixed_flux_comparison_2026-04-28.md` — Mixed-Flux comparison

10. SAFS Mesh Pipeline (**safs/**)

The San Andreas Fault System target. Not yet wired into a runtime driver — currently a Python preprocessing + meshing pipeline producing Gmsh `.msh` files for future SEAS / dynamic rupture runs.

```
safs/
├── CFM_data_step/                # SCEC CFM fault surfaces (.step) at 500/1000/2000 m
├── project_7.0_alternative/      # NW-cut (hard-clip) approach
│   ├── code_preprocess/         # ts_to_stl, retriangulate, NW cut, free-surface clean
│   ├── code_meshing/           # gmsh .geo templates + driver scripts
│   ├── data_*                   # intermediate / output meshes per stage
│   └── document/PLAN_nw_hard_cut.md
└── project_7.0_preferred/       # CGAL corefine multi-fault approach (current default)
    ├── raw_data/               # Mission Creek, Garnet Hill, Banning, SAF strands
    ├── code_preprocess/        # ts_to_stl, corefine_faults, clean_freesurfacemesh,
    │                           # sensitivity_sweep, generate_multifault_geo
    ├── code_meshing/           # safs_multifault_box_2000m.geo
    ├── data_corefined/         # CGAL-corefined fault surfaces
    ├── data_cleanfreesurf/     # meshes with cleaned topography
    └── document/PLAN_cgal_corefine_multifault*.md
```

Pipeline (preferred / CGAL-corefine):

1. `ts_to_stl.py` — convert SCEC CFM `.ts` triangulated surfaces to `.stl`.
2. `corefine_faults.py` — CGAL polygon-mesh corefine across multiple fault strands (resolves intersections without losing geometry).
3. `clean_freesurface_mesh.py` — trim above-surface artefacts, enforce $z = 0$ topography continuity.
4. `generate_multifault_geo.py` -> `safs_multifault_box_2000m.geo` (Gmsh `.geo` with embedded fault surfaces inside a Cartesian box domain).
5. Gmsh -> `.msh`. Quality check via `check_msh_quality.py` and `sensitivity_sweep.py`.

Environment: `conda activate pythonenv` for all SAFS Python tooling.

Status: see `document/system_dev/` plans and any `STATUS.md` on the SAFS branch (feature/safs-quasi-dynamic).

11. Parallelism & MPI

File	Purpose
common/mpi_context.hpp	RAII MPI wrapper. GlobalMax, GlobalSum, GatherToRoot, Barrier, Bcast (no-ops when SEAS_USE_MPI is off).
common/fault_scatter.hpp	Owned-<->-ghost fault DOF exchange (non-blocking point-to-point).
common/parallel_utils.hpp	Generic gather / scatter helpers.
common/mpi_check.hpp, mpi_tags.hpp	Error-checking macros + tag table.
dynamic/fault_locality_partition.hpp	Constrains MPI partition so both elements of every fault face are co-resident on the same rank (eliminates dip-direction pollution at $np \geq 4$ on y-mirror meshes).

MPI surface inside one **Mult** call (quasi-dynamic):

- Step 2a — FaultScatter (owned slip -> ghost DOFs)
- Step 2b — ParBilinearForm::Assemble (first call only) and parallel sparse solve (MUMPS / Hypre BoomerAMG)
- Post-stage — MPI_Allreduce(MAX) on the error norm (consistent accept/reject)
- Post-stage — MPI_Allreduce(MAX) on $V_{\max}$ (earthquake detection / dt control)

Critical safety rule: the error-norm reduction prevents rank divergence; without it ranks deadlock inside the next collective in Solve(). See CODEBASE_GUIDE.md §11 for the flowchart.

User preference: 8 MPI ranks for local BP5 runs.

12. Build & Test Reference

Environments

```
conda activate mfem-dev      # build + run (mpicxx, MPI, MUMPS, Hypre, PETSc)
conda activate pythonenv     # SAFS / Gmsh Python tooling
```

Top-level targets

```
cd miniapps/seas
make all                # everything in $(MINIAPPS)
make seas_driver        # quasi-dynamic TOML driver (BP5 default)
make seas_pseas         # legacy BP2 antiplane
make seas_bp5_full      # BP5 verification driver
make seas_tpv102_driver  # SCEC TPV102 dynamic
make seas_tpv104_driver  # SCEC TPV104 dynamic
make seas_tpv205_driver  # SCEC TPV5/205 dynamic
```

Test matrix

Layer	Location	Run via	Indicative count
Unit (serial)	tests/unit/	make test, make test-<name>	~80+
Parallel (MPI)	tests/parallel/	mpirun -np N seas_test-<name>	~25+
Verification (long)	tests/verification/	mpirun -np N seas-<bench>_full --mesh ...	7

Common targets:

```

make test                                # full unit-test sweep (~2 min)
make test-friction                       # friction law unit tests
make test-elasticity-operator            # 3D DG elasticity
make test-bp5-integration                # BP5 integration test (serial)
make test-bp5-smoke                     # -> seas_test_bp5_parallel_smoke (4 ranks)

# Parallel
mpirun -np 4 seas_test_parallel_elasticity
mpirun -np 8 seas_test_bp5_parallel_smoke

# Verification (long-running)
mpirun -np 8 seas_bp5_full --mesh bp5/mesh/bp5_1000m.msh --tfinal 56844000000
mpirun -np 8 seas_tpv102_driver --mesh tpv102/mesh/tpv102_200m.msh --tfinal 12.0 --ader-order 2

# Dynamic-rupture driver examples
ibrun ./seas_tpv104_driver --mesh tpv104/mesh/tpv104_200m.msh \
--tfinal 3.0 --ader-order 2 \
--friction-solver newton-stable \
--output-dir tpv104/results

```

jobs/<benchmark>/*.sbatch holds the Frontera SLURM scripts.

Regression flags (anything below means the code broke)

1. $V_{\max}$ monotonically increasing (should peak then decay through events).
2. Traction > 1 GPa for $\sigma_n = 25$ MPa (physical limit).
3. $\Delta t \rightarrow 0$ or NaN.
4. zeroIn bracket failure ($F(a)$ and $F(b)$ same sign).
5. Recurrence $\gg 300$ yr or $\ll 100$ yr (BP5 expected ~ 240 yr).
6. Event slip $\ll 1$ m or $\gg 10$ m.
7. Significant dip slip in BP5 (should be ≈ 0 for pure strike-slip).

13. I/O, Output & Visualization

File	Format / role
io/probe_output.hpp	Single-station ASCII (e.g. global $V_{\max}$ log)
io/benchmark_output.hpp, parallel_benchmark_output.hpp	Generic benchmark probe writers
io/bp5_benchmark_output.hpp, bp5_parallel_output.hpp	SCEC BP5 fltst distributed format (per-station owner rank)
io/paraview_output.hpp	VTK / ParaView snapshots (bulk + fault surface)
io/checkpoint.hpp	Binary checkpoint / restart
io/data_field_3d.{hpp,cpp}	HDF5 schema-v1 sidecar reader + trilinear interpolator (data-projection feature)
io/field_coefficient.{hpp,cpp}	mfem::Coefficient wrapper for DataField3D + FieldProjector
trace/face_trace_logger.hpp	Per-face runtime tracer — rank-local, append-only, no MPI inside (debugging)

The data-projection sidecar contract (CRS = EPSG:32611, z positive = elevation, etc.) is in document/features_dev/data_projection

14. Files Requiring Extreme Care

Editing any file below mandates: read its `debug_document/` history first, run the full unit + relevant verification suite after, and (per `CLAUDE.md`) do not revert a prior fix without citing the debug doc and getting explicit approval.

File	Why
<code>friction/dieterich_ruina.hpp</code>	Brent solver, log-V bracket; wrong tolerance -> silent wrong V
<code>domain/elasticity_operator.hpp</code>	DG assembly, traction, BCs; impacts every sim
<code>fault/fault_basis.hpp</code>	Sign chain — error here -> positive feedback -> blow-up
<code>fault/rate_state_fault.hpp</code>	State interleaving and ODE-RHS construction
<code>solver/seas_operator.hpp</code>	Coupling and 4-phase init
<code>solver/time_stepper.hpp</code>	Parallel error reduction is critical for MPI safety
<code>config/bp5_params.hpp</code>	Spatial $a(z)$, D_c — wrong -> wrong friction regime
<code>dynamic/fault_face_flux.{hpp,cpp}</code>	Shared by TPV102/104/205 + BP5 dynamic
<code>dynamic/wave_operator.{hpp,inl}</code>	Shared bulk operator + ADER predictor

Project-wide feedback rules (do not violate):

- Follow SCEC benchmark spec exactly rather than matching Tandem's deviations.
- Never hardcode numerical constants — derive from parameters / mesh.
- Audit all occurrences of a pattern before changing it (interior + shared faces, all output files).
- Write implementation plans into `debug_document/`<problem>_debug_document/, not into ad-hoc plan files.

15. Document Map (Where Things Live)

Topic	Document
Class hierarchy + per-stage execution flow	ARCHITECTURE.md
Physics -> numerics -> code, with full DG-IP derivation	CODEBASE_GUIDE.md
Critical numerical constraints + extreme-care list	CLAUDE.md
High-level component map (this file)	document/codebase_explorer/DEVELOPER_GUIDE.md
BP1/BP2 / BP5 debug history	debug_document/{bp1_bp2,bp5}_debug_document/
TPV102 / TPV104 debug history	debug_document/tpv10{2,4}_debug_document/
BP5 refactoring history	document/system_dev/bp5_refactoring_plan_v{1..4}.md
Dynamic rupture design	document/system_dev/dynamic_rupture_plan_v{1..4}.md
Unified framework / cycle wiring	document/system_dev/unified_framework_and_cycle_wiring_plan_v{1..4}.md
Mixed-Flux DG comparison	document/system_dev/drdg3d_mixed_flux_comparison_2026-04-28.md
TPV205 LSW design	document/system_dev/tpv205_lsw_native_fields_plan_2026-04-27.md
Data-projection feature	document/features_dev/data_projection_*.md
Anti-plane / SBI / full-elasticity development notes	document/{antiplane_dev, sbi_dev, fullelasticity_dev}/
Per-benchmark gold reference data	<bench>/benchmark_data/, <bench>/gold/
SLURM job scripts (Frontera)	jobs/<bench>/*.sbatch
SAFS plans	safs/project_7.0_*/document/PLAN_*.md

Building this guide as PDF

```
cd miniapps/seas/document/codebase_explorer
pandoc DEVELOPER_GUIDE.md -o DEVELOPER_GUIDE.pdf \
  --pdf-engine=xelatex --toc --toc-depth=2 \
  --highlight-style=tango -V colorlinks=true
open DEVELOPER_GUIDE.pdf
```

16. Change Log

Version	Date	Author	Summary
v0.1	2026-05-09	initial draft	First DEVELOPER_GUIDE: covers all current modules including the post-CODEBASE_GUIDE.md-v3 additions (dynamic/, TPV102/104/205 drivers, safes/, trace/, data-projection I/O). Quick-access cheatsheet, symbol glossary, document map, build & test reference, change log. Awaiting reviewer feedback.
v0.2	TBD	—	<i>Reserved for first round of reviewer suggestions.</i>

How to revise this document

1. Bump the version in the YAML front-matter `date:` field and in this change log.
2. Add a new row at the top of the change log describing the diff (what was added / removed / renamed).
3. If a section grows beyond ~one screen, consider splitting it into a sibling file under `document/codebase_explorer/` and linking from §15.
4. Re-render PDF (`pandoc ... -o DEVELOPER_GUIDE.pdf`).
5. Commit message convention: `developer-guide: vX.Y - <one-line summary>`.

Maintenance trigger: update this guide whenever a new top-level directory is added to `miniapps/seas/`, a new driver is created, a module moves, or an “extreme-care” file changes status.